

Tài liệu Đồ án Môn học SE360: Xây dựng Nền tảng "UIT-Go" Cloud-Native

1. Tổng quan

Chào mừng các bạn đến với đồ án môn học SE360. Trong bối cảnh ngành công nghiệp phần mềm đang dịch chuyển mạnh mẽ sang kiến trúc cloud-native, vai trò của người kỹ sư không còn chỉ gói gọn trong việc viết code. Các công ty công nghệ hàng đầu hiện nay tìm kiếm những chuyên gia có "T-shaped skills" - những người không chỉ sâu về một lĩnh vực mà còn có hiểu biết rộng về toàn bộ vòng đời của sản phẩm.

Mục tiêu của đồ án này không chỉ là xây dựng một ứng dụng, mà là một hành trình mô phỏng vai trò của một Kỹ sư Hệ thống (System Engineer) tại một công ty công nghệ. Đây là cơ hội để các bạn xây dựng một sản phẩm 'portfolio-defining' (dự án làm nên thương hiệu cá nhân), giải quyết các bài toán thực tế: **làm thế nào để thiết kế, triển khai, và vận hành một hệ thống phân tán có khả năng mở rộng, đáng tin cậy, và an toàn?** Đây chính là những kỹ năng cốt lõi giúp bạn khác biệt và được các công ty công nghệ hàng đầu săn đón. Việc thấu hiểu các đánh đổi (trade-offs) giữa chi phí, hiệu năng, và độ phức tạp sẽ là kim chỉ nam cho mọi quyết định của bạn trong đồ án này.

Chúng ta sẽ cùng nhau giải quyết bài toán xây dựng hệ thống backend cho **UIT-Go**, một ứng dụng gọi xe giả tưởng. Thay vì xây dựng tất cả các tính năng, đồ án sẽ được chia làm hai giai đoạn chính:

1. **Giai đoạn 1: Xây dựng "Bộ Xương" Microservices:** Tất cả các nhóm sẽ xây dựng một nền tảng microservices tối thiểu, đóng vai trò là khung sườn của hệ thống.
2. **Giai đoạn 2: Module Chuyên sâu:** Mỗi nhóm sẽ **chọn MỘT** trong năm lĩnh vực chuyên sâu để nghiên cứu và hiện thực hóa. Đây là cơ hội để các bạn đi sâu vào một thách thức kỹ thuật cụ thể của các hệ thống hiện đại và định hình vai trò chuyên môn của mình.

2. Bối cảnh & Yêu cầu Nghiệp vụ

UIT-Go là một ứng dụng cho phép hành khách đặt xe và kết nối với các tài xế ở gần. Hệ thống cần xử lý các luồng nghiệp vụ cơ bản sau:

- **Hành khách:**
 - *User Story 1:* Là một người dùng mới, tôi muốn có thể đăng ký tài khoản bằng email và mật khẩu để sử dụng ứng dụng.
 - *User Story 2:* Là một hành khách, tôi muốn nhập điểm đi và điểm đến để có thể yêu cầu một chuyến xe và xem trước giá cước ước tính.
 - *User Story 3:* Trong lúc chờ xe, tôi muốn thấy được vị trí của tài xế đang di chuyển trên bản đồ theo thời gian thực để biết khi nào họ sẽ tới.
 - *User Story 4:* Là một hành khách, tôi muốn có thể hủy chuyến đi nếu có việc đột xuất.

- *User Story 5*: Sau khi chuyến đi kết thúc, tôi muốn có thể đánh giá tài xế (từ 1-5 sao) và để lại bình luận để phản hồi về chất lượng dịch vụ.
- **Tài xế:**
 - *User Story 1*: Là một tài xế, tôi muốn có thể đăng ký thông tin cá nhân và phương tiện để được xét duyệt tham gia hệ thống.
 - *User Story 2*: Khi bắt đầu ca làm việc, tôi muốn bật trạng thái "Sẵn sàng" (Online) để bắt đầu nhận các yêu cầu chuyến đi mới.
 - *User Story 3*: Khi có yêu cầu chuyến đi ở gần, tôi muốn nhận được thông báo và có 15 giây để quyết định chấp nhận hay từ chối.
 - *User Story 4*: Trong suốt quá trình di chuyển đến điểm đón và chờ khách, vị trí của tôi cần được cập nhật liên tục về hệ thống.
 - *User Story 5*: Khi đến nơi, tôi muốn có thể bấm nút "Hoàn thành" để kết thúc chuyến đi và hệ thống ghi nhận doanh thu.

3. Giai đoạn 1: "Bộ Xương" Microservices (Bắt buộc cho tất cả các nhóm)

Mục tiêu của giai đoạn này là xây dựng nền tảng vững chắc cho UIT-Go, nơi các service có thể giao tiếp với nhau một cách tin cậy.

3.1. Kiến trúc yêu cầu

Mỗi nhóm cần xây dựng và triển khai ít nhất 3 microservices cơ bản sau:

1. **UserService:**
 - **Trách nhiệm:** Quản lý thông tin người dùng (hành khách và tài xế), xử lý đăng ký, đăng nhập và hồ sơ.
 - **API gợi ý:** POST /users, POST /sessions (login), GET /users/me.
 - **Công nghệ gợi ý:** Node.js/Go/Python, PostgreSQL/MySQL (RDS).
2. **TripService:**
 - **Trách nhiệm:** Dịch vụ trung tâm, xử lý logic tạo chuyến đi, quản lý các trạng thái của chuyến (đang tìm tài xế, đã chấp nhận, đang diễn ra, hoàn thành, đã hủy).
 - **API gợi ý:** POST /trips, GET /trips/{id}, POST /trips/{id}/cancel.
 - **Công nghệ gợi ý:** Node.js/Go/Python, PostgreSQL/MongoDB (RDS/DocumentDB).
3. **DriverService:**
 - **Trách nhiệm:** Quản lý trạng thái và vị trí của tài xế theo thời gian thực. Cung cấp API để tìm kiếm các tài xế phù hợp ở gần.
 - **API gợi ý:** PUT /drivers/{id}/location, GET /drivers/search?lat=...&lng=....
 - **Công nghệ gợi ý:** Node.js/Go/Python. Đối với dữ liệu vị trí, hãy phân tích và lựa chọn giữa hai hướng tiếp cận chính:
 - **Ưu tiên tốc độ (Speed-first):** Sử dụng Redis (ElastiCache) với tính năng Geospatial để tìm kiếm tài xế trong bán kính gần với độ trễ cực thấp.
 - **Ưu tiên khả năng mở rộng và chi phí (Scale/Cost-first):** Sử dụng DynamoDB kết hợp với Geohashing để tối ưu cho lượng ghi (write) vị trí khổng lồ và chi phí

vận hành khi hệ thống lớn mạnh.

3.2. Yêu cầu kỹ thuật cho "Bộ Xương"

- **Giao tiếp:** Các service giao tiếp với nhau qua API. Hãy phân tích và lựa chọn giữa RESTful (đơn giản, phổ biến) và gRPC (hiệu năng cao, phù hợp cho giao tiếp nội bộ).
- **Containerization:** Toàn bộ các service phải được đóng gói bằng Docker và có thể chạy độc lập thông qua Docker Compose trên môi trường local.
- **Cơ sở dữ liệu:** Mỗi service phải có cơ sở dữ liệu riêng, tuân thủ nguyên tắc "Database per Service" để đảm bảo sự độc lập và khả năng mở rộng.
- **Infrastructure as Code (IaC):** Sử dụng Terraform để định nghĩa và triển khai các tài nguyên hạ tầng cơ bản (VPC, Subnets, Security Groups, IAM Roles, Databases). Việc này đảm bảo hạ tầng của bạn có thể được tạo lại một cách nhất quán và được quản lý phiên bản như code.
- **Triển khai:** Các container được triển khai trên AWS. Hãy cân nhắc giữa việc tự quản lý trên EC2 (linh hoạt tối đa) và sử dụng các dịch vụ điều phối như ECS/EKS (giảm gánh nặng vận hành).

4. Giai đoạn 2: Lựa chọn Module Chuyên sâu (Chọn 1 trong 5)

Sau khi hoàn thành "bộ xương", mỗi nhóm sẽ chọn một module để đi sâu và hoàn thiện hệ thống của mình. Mỗi module đều có độ khó và chiều sâu tương đương, đòi hỏi nhóm phải phân tích, bảo vệ các quyết định thiết kế và đánh đổi của mình.

Module A: Thiết kế Kiến trúc cho Scalability & Performance

- **Vai trò:** Kỹ sư Kiến trúc Hệ thống (System Architect).
- **Mục tiêu:** Chủ động **thiết kế một kiến trúc có khả năng đạt tới hyper-scale**, thay vì chỉ tinh chỉnh (tuning) hệ thống hiện có. Tập trung vào việc phân tích và ra quyết định đối với các lựa chọn kiến trúc nền tảng và các **đánh đổi (trade-off)** quan trọng để đảm bảo trải nghiệm người dùng mượt mà khi quy mô tăng vọt.
- **Nhiệm vụ cụ thể:**
 1. **Phân tích và Bảo vệ Lựa chọn Kiến trúc:** Phân tích các luồng nghiệp vụ quan trọng (tìm kiếm tài xế, cập nhật vị trí), từ đó đề xuất và bảo vệ các quyết định thiết kế nền tảng. Ví dụ: "Chúng em sử dụng mô hình giao tiếp bất đồng bộ với SQS giữa TripService và DriverService. Điều này giúp hệ thống chịu được lượng yêu cầu đặt xe tăng đột biến mà không làm sập DriverService, nhưng đánh đổi là độ trễ tìm xe sẽ tăng nhẹ."
 2. **Kiểm chứng Thiết kế bằng Load Testing:** Xây dựng và thực thi các kịch bản load testing (sử dụng k6, JMeter...) để kiểm chứng giả định thiết kế, tìm ra điểm nghẽn cổ chai (bottleneck) và đo lường giới hạn của hệ thống (ví dụ: hệ thống chịu được bao nhiêu request/giây).
 3. **Hiện thực hóa các Kỹ thuật Tối ưu hóa (Tuning):** Dựa trên kết quả, áp dụng

caching (ElastiCache) cho các dữ liệu ít thay đổi, thiết lập Auto Scaling Group cho các service, và thực hiện các chiến lược mở rộng CSDL như Read Replicas.

- **Sản phẩm đầu ra:** Báo cáo chuyên sâu, trong đó phải có một phần riêng biệt **phân tích các lựa chọn thiết kế và các trade-off đã thực hiện** (ví dụ: Consistency vs. Availability, Cost vs. Performance), kèm theo biểu đồ kết quả load testing trước và sau khi tối ưu.

Module B: Thiết kế cho Reliability & High Availability

- **Vai trò:** Kỹ sư Đảm bảo Độ tin cậy (Site Reliability Engineer - SRE).
- **Mục tiêu:** Thiết kế một hệ thống có khả năng chống chịu và tự phục hồi trước các sự cố, thay vì chỉ cấu hình các tính năng sẵn sàng cao. Tập trung vào việc phân tích các điểm có thể gây lỗi (single point of failure), định lượng độ tin cậy, và thực hành các kịch bản sự cố.
- **Nhiệm vụ cụ thể:**
 1. **Phân tích và Loại bỏ Điểm lỗi:** Vẽ sơ đồ kiến trúc, xác định các điểm có thể gây lỗi và đề xuất giải pháp kiến trúc để loại bỏ chúng. Ví dụ: triển khai các service trên nhiều Availability Zone (Multi-AZ), sử dụng Application Load Balancer để phân tải và tự động chuyển hướng traffic khi một node bị lỗi.
 2. **Thực hành Chaos Engineering:** Sử dụng các công cụ như AWS Fault Injection Simulator để chủ động "tiêm" lỗi vào hệ thống (ví dụ: tắt một service, làm tăng latency của CSDL) và phân tích hành vi của hệ thống để kiểm chứng các pattern tăng độ tin cậy (Retry, Circuit Breaker, Timeout).
 3. **Thiết kế và Thực hành Kịch bản Phục hồi sau Thảm họa (DR):** Xây dựng một quy trình DR chi tiết, tính toán RTO (Recovery Time Objective) và RPO (Recovery Point Objective) cho hệ thống. Thực hành việc phục hồi toàn bộ hệ thống sang một Region khác sử dụng các bản sao lưu CSDL (snapshots) và hạ tầng IaC.
- **Sản phẩm đầu ra:** Báo cáo phân tích các điểm lỗi và giải pháp, video/log ghi lại quá trình thực hành Chaos Engineering và kịch bản DR thành công, trình bày về các **trade-off đã thực hiện**. Ví dụ: "Chúng em đã cân nhắc giữa kiến trúc CSDL Single-AZ và Multi-AZ. Phương án Single-AZ có chi phí thấp hơn 50% và độ trễ ghi (write latency) thấp hơn khoảng 5-10ms. Tuy nhiên, RTO (Recovery Time Objective) khi xảy ra sự cố là 15-30 phút. Ngược lại, Multi-AZ tăng độ tin cậy lên 99.99% và RTO gần như bằng 0, nhưng chi phí tăng gấp đôi. Sau khi phân tích, chúng em quyết định chọn Multi-AZ vì tính sẵn sàng của dịch vụ đặt xe là yêu cầu nghiệp vụ cốt lõi, và chúng em chấp nhận đánh đổi về chi phí và một chút hiệu năng."

Module C: Thiết kế cho Security (DevSecOps)

- **Vai trò:** Kỹ sư Bảo mật (Security Engineer).
- **Mục tiêu:** Thiết kế và xây dựng một hệ thống an toàn theo triết lý "Zero Trust" (không tin tưởng bất kỳ ai), thay vì chỉ cấu hình các công cụ bảo mật. Tập trung vào việc phân tích các mối đe dọa, thiết kế các lớp phòng thủ sâu (defense-in-depth) và tích hợp bảo mật vào quy trình phát triển (Shift-left security).
- **Nhiệm vụ cụ thể:**
 1. **Mô hình hóa Mối đe dọa (Threat Modeling):** Xây dựng Sơ đồ Luồng Dữ liệu (Data

Flow Diagram), sau đó phân tích kiến trúc hệ thống để xác định các bề mặt tấn công tiềm tàng và các mối đe dọa (sử dụng mô hình STRIDE), từ đó đề xuất các biện pháp giảm thiểu.

2. **Thiết kế Kiến trúc Mạng Zero Trust:** Thiết kế và cấu hình VPC với các public và private subnets, Network ACLs, Security Groups một cách chặt chẽ để cô lập các service và chỉ cho phép các luồng giao tiếp tối thiểu cần thiết.
 3. **Xây dựng Vòng đai Bảo mật Dữ liệu và Định danh:** Hiện thực hóa luồng xác thực an toàn (với Cognito), áp dụng nguyên tắc đặc quyền tối thiểu (Least Privilege) cho tất cả IAM Roles, mã hóa dữ liệu cả khi đang truyền (TLS) và khi lưu trữ (KMS), và quản lý an toàn secrets (với Secrets Manager).
- **Sản phẩm đầu ra:** Một bản báo cáo Threat Modeling, sơ đồ kiến trúc mạng chi tiết, và trình bày về các **trade-off giữa mức độ bảo mật và sự thuận tiện cho việc phát triển/vận hành**. Ví dụ: "Việc bắt buộc mọi truy cập CSDL phải thông qua một bastion host làm tăng tính an toàn nhưng cũng làm tăng độ phức tạp cho developer khi cần gỡ lỗi."

Module D: Thiết kế cho Observability

- **Vai trò:** Kỹ sư Vận hành (Platform/Operations Engineer).
- **Mục tiêu:** Thiết kế một hệ thống có khả năng "tự chẩn đoán", cung cấp thông tin sâu sắc về trạng thái và hiệu năng, thay vì chỉ thu thập logs và metrics. Tập trung vào việc xác định các chỉ số đo lường quan trọng và kết nối chúng với trải nghiệm người dùng và mục tiêu kinh doanh.
- **Nhiệm vụ cụ thể:**
 1. **Định nghĩa SLOs/SLIs:** Xác định các **Mục tiêu Mức độ Dịch vụ (SLOs)** và **Chỉ số Mức độ Dịch vụ (SLIs)** quan trọng cho các luồng nghiệp vụ chính (ví dụ: tỷ lệ đặt xe thành công > 99.9%, độ trễ API tìm tài xế ở phân vị 95 (p95) < 200ms). Định nghĩa "Error Budget" cho từng SLO.
 2. **Xây dựng Nền tảng Quan sát:** Thiết lập hệ thống logging tập trung (CloudWatch Logs), tích hợp metrics tùy chỉnh để theo dõi SLIs, và triển khai Distributed Tracing (AWS X-Ray) để có thể theo dõi hành trình của một request qua nhiều microservices.
 3. **Xây dựng Dashboard và Hệ thống Cảnh báo Thông minh:** Thiết kế một Dashboard trực quan hóa các SLIs/SLOs và Error Budget. Cấu hình hệ thống cảnh báo (Alarms) tự động khi có nguy cơ vi phạm SLO và tạo ra một "runbook" hướng dẫn cách xử lý sự cố cho một cảnh báo cụ thể.
- **Sản phẩm đầu ra:** Một Dashboard giám sát hoàn chỉnh theo SLOs/SLIs, trình bày khả năng truy vết một request phức tạp bằng X-Ray, và báo cáo phân tích các **trade-off khi lựa chọn giữa các loại dữ liệu quan sát (logs, metrics, traces)**.

Module E: Thiết kế cho Automation & Cost Optimization (FinOps)

- **Vai trò:** Kỹ sư Nền tảng & Tối ưu hóa (Platform & FinOps Engineer).
- **Mục tiêu:** Thiết kế một quy trình phát triển và vận hành hiệu quả, có khả năng kiểm soát và tối ưu hóa chi phí, thay vì chỉ xây dựng pipeline và cắt giảm chi phí một cách thụ động. Tập trung vào việc xây dựng một "nền tảng" tự phục vụ cho developer và áp dụng

các nguyên tắc quản lý tài chính trên đám mây.

- **Nhiệm vụ cụ thể:**

1. **Thiết kế Nền tảng Tự phục vụ (Self-Service Platform):** Xây dựng một pipeline CI/CD hoàn chỉnh (sử dụng GitHub Actions) và cấu trúc lại code Terraform theo dạng modules có thể tái sử dụng, cho phép một developer mới có thể triển khai một service một cách an toàn và nhanh chóng.
 2. **Xây dựng Cơ chế Quản lý Chi phí:** Phân tích chi phí hệ thống bằng AWS Cost Explorer. Thiết lập cơ chế **gắn thẻ (tagging)** tài nguyên một cách nhất quán để có thể phân bổ chi phí theo từng service. Cấu hình AWS Budgets để gửi cảnh báo khi chi phí có nguy cơ vượt ngưỡng.
 3. **Nghiên cứu và Bảo vệ Quyết định Tối ưu:** Đề xuất và phân tích các phương án tối ưu chi phí (Spot Instances, Serverless, Graviton processors...). Hiện thực hóa một phương án và đo lường hiệu quả bằng số liệu cụ thể.
- **Sản phẩm đầu ra:** Trình bày một quy trình CI/CD tự động, mã nguồn Terraform được module hóa, và một báo cáo phân tích chi phí, trong đó nhấn mạnh các **trade-off giữa chi phí, hiệu năng và công sức vận hành**.

5. Yêu cầu Nộp bài & Tiêu chí Đánh giá

5.1. Sản phẩm cần nộp

1. **Mã nguồn:** Link repository GitHub công khai, có cấu trúc chuyên nghiệp.
2. **Tài liệu:**
 - README.md: Rõ ràng, hướng dẫn cách cài đặt và chạy hệ thống trên môi trường local và trên AWS.
 - ARCHITECTURE.md: Sơ đồ kiến trúc hệ thống tổng quan và sơ đồ chi tiết cho module chuyên sâu.
 - ADR/: Một thư mục chứa các bản ghi quyết định kiến trúc (Architectural Decision Records). Mỗi file markdown ghi lại một quyết định quan trọng (ví dụ: tại sao chọn Redis thay vì DynamoDB, tại sao chọn gRPC thay vì REST) cùng bối cảnh và các đánh đổi đã cân nhắc. Đây là bằng chứng cho quá trình tư duy thiết kế của nhóm.
 - REPORT.md: Báo cáo chuyên sâu (3-5 trang) theo cấu trúc:
 1. **Tổng quan kiến trúc hệ thống:** Sơ đồ và giải thích ngắn gọn.
 2. **Phân tích Module chuyên sâu:** Mô tả cách tiếp cận và kết quả.
 3. **Tổng hợp Các quyết định thiết kế và Trade-off (Quan trọng nhất):** Đây là phần cốt lõi của báo cáo, tổng hợp lại những điểm chính từ các ADR của nhóm. Cần trình bày rõ các lựa chọn đã cân nhắc, tại sao lại chọn giải pháp hiện tại và đã phải đánh đổi những gì (về chi phí, hiệu năng, độ phức tạp...).
 4. **Thách thức & Bài học kinh nghiệm:** Những khó khăn kỹ thuật đã gặp và bài học rút ra.
 5. **Kết quả & Hướng phát triển:** Tóm tắt kết quả và đề xuất cải tiến trong tương lai.
3. **Video Demo (5-7 phút):** Quay lại màn hình demo các chức năng chính và các tính năng của module chuyên sâu.

5.2. Buổi Báo cáo Cuối kỳ (15 phút/nhóm)

- **Demo trực tiếp (5 phút):** Vận hành các luồng chính của hệ thống.
- **Trình bày kiến trúc & Module chuyên sâu (7 phút):** Giải thích các lựa chọn thiết kế và trình bày kết quả của module chuyên sâu.
- **Q&A (3 phút).**

5.3. Tiêu chí đánh giá

- **Hoàn thiện "Bộ Xương" (30%):** Mức độ hoàn thiện, ổn định và chất lượng triển khai của các microservices cơ bản.
- **Module Chuyên sâu (40%):**
 - **Độ sâu Phân tích & Thiết kế (20%):** Khả năng phân tích vấn đề, đề xuất giải pháp và lý giải các trade-off.
 - **Chất lượng Hiện thực hóa (20%):** Mức độ hoàn thiện của các giải pháp kỹ thuật và kết quả đạt được.
- **Chất lượng Kỹ thuật (15%):** Chất lượng mã nguồn (code, IaC), cấu trúc repository, tài liệu (đặc biệt là các ADR).
- **Báo cáo & Trình bày (15%):** Khả năng trình bày vấn đề một cách rõ ràng, súc tích, demo và trả lời câu hỏi.

6. Lộ trình Đồ án & Các Cột mốc

Đây là lộ trình gợi ý, các nhóm có thể điều chỉnh cho phù hợp.

Tuần	Hoạt động chính	Lý thuyết tương ứng (trong giáo trình)
1-2	Khởi động & Thiết kế: Thành lập nhóm, đăng ký module chuyên sâu, phân tích yêu cầu, vẽ sơ đồ kiến trúc tổng quan, thiết lập repository và quy trình làm việc.	Chương 1, 2: Tư duy Hệ thống, Microservices
3-5	Xây dựng "Bộ Xương": Code 3 microservices, thiết kế schema CSDL, viết Dockerfile. Chạy thử nghiệm trên local bằng Docker Compose.	Chương 3, 4, 5: Dữ liệu, 12-Factor App, Containers
6-7	Triển khai Hạ tầng Cơ	Chương 6, 7, 8: Mạng, IAM,

	bản: Viết Terraform cho VPC, DB, IAM. Bắt đầu triển khai các services lên AWS.	laC
8	CỘT MỐC 1: BÁO CÁO TIẾN ĐỘ: Demo "bộ xương" chạy được trên môi trường local (sử dụng Docker Compose), trong đó các service có thể giao tiếp thành công với nhau qua API (ví dụ: chứng minh UserService có thể gọi TripService). Nộp phiên bản đầu tiên của file ARCHITECTURE.md và trình bày kế hoạch chi tiết cho module chuyên sâu.	-
9-12	Hiện thực hóa Module Chuyên sâu: Tập trung vào các nhiệm vụ của module đã chọn. Liên tục cập nhật các quyết định vào ADR.	Chương 9-14: Observability, Caching, Reliability, Security, Cost
13	Hoàn thiện & Tổng kết: Tích hợp, kiểm thử toàn bộ hệ thống. Hoàn thiện code, viết báo cáo, chuẩn bị slide, quay video demo.	-
14	CỘT MỐC 2: BÁO CÁO CUỐI KỲ: Nộp sản phẩm và báo cáo trước lớp.	Chương 15: Tổng kết

Chúc các bạn có một kỳ đồ án thành công và học hỏi được nhiều kiến thức bổ ích!